Dynamic Programming

Longest Common Subsequence

Reading

CLR section 15.4

Longest Common subsequence

- **Input:**
  - Two sequences of numbers $X = x_1, \ldots, x_n$ and $Y = y_1, \ldots, y_n$
- **Output:**
  - The **Longest Common Subsequence** of the X and Y

LCS: Definitions and Examples

- **Sequence**: ordered list of elements
- Z is a **subsequence** of X if Z can be obtained from X by deleting some elements
  - Example: X = ABCDEFGH
    - Z = ADF is a subsequence of X
    - Z = BD is another subsequence of X
- Z is a **common subsequence** of X and Y is Z is a subsequence of both X and Y
- Z is a **longest common subsequence** of X and Y
  - Any sequence longer than Z is not a subsequence of at least one of X or Y
- **Example:**
  - X=ABCBDAB Y=BDCABA
  - Common subsequence: A, AB
  - Longest common subsequences: BCBA, BCAB, BDAB
    - LCS is not unique, so we want to find a LCS not the LCS

LCS: Brute-force algorithm

- Algorithm:
  - Check every subsequence of X[1..m] to see if it is also a subsequence of Y[1..n]
- Runtime Analysis:
  - Number of subsequences of X: 2^m
  - Time to check if a subsequence of X is also a subsequence of Y: O(n+m)
  - Runtime = $O((n+m).2^m)$

LCS: Towards a better algorithm

Simplification:

1) Find the **length** of a longest-common subsequence
2) Extend the algorithm to find the LCS

Notations:

Length of a sequence: |X|

Notation: X_i for a given string X denotes the prefix of length i: $x_1 , \ldots , x_i$.

$X = x_1 x_2 \ldots x_m$

$Y = y_1 y_2 \ldots y_n$

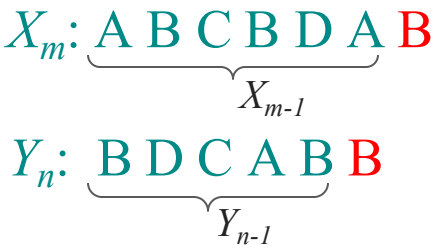
X_i = prefix of length i in string X = $X[1 \ldots i] = x_1 x_2 \ldots x_i$

Y_j = prefix of length j in string Y = $Y[1 \ldots j] = y_1 y_2 \ldots y_j$

`C[i, j] = length of LCS(`X_i, Y_j`)`

LCS: Optimal Substructure

- **Case1: $x_m = y_n$** and **Z= LCS(X, Y)**



  - If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1}= LCS(X_{m-1},Y_{n-1})
  - **Proof:**
    - If $z_k \neq x_m$ then append x_m to z to get a LCS of length k+1 → contradiction
  - Proving Z_{k-1} **is a Longest CS of** X_{m-1},Y_{n-1}
    - Z_{k-1}= **CS**(X_{m-1},Y_{n-1}) (it is a common subsequence). Why**?**
    - Z_{k-1} is also a **LCS**(X_{m-1},Y_{n-1})
      - if not we can replace it with a longer common subsequence and get a LCS (X, Y) which is longer than Z. Contradiction

LCS: Optimal Substructure

- **Case2: $x_m \neq y_n$ and Z = LCS(X, Y)**
  - If $x_m \neq y_n$ and $z_k \neq x_m$, then $Z = LCS(X_{m-1}, Y)$
  - **Proof:**
    - If $z_k \neq x_m$ then Z is a common subsequence of $\mathbf{X_{m-1}}$ and **Y.**
    - Z **is a Longest CS of** X_{m-1}, Y_n
      - if not, there would be another common subsequence of $\mathbf{X_{m-1}}$ and **Y.** Let's name it W. Then W is longer than Z and it is a common subsequence of X and Y.
      - →Contradiction. Z was the longest common subsequence

X_m: A B C B D A D

X_{m-1}

Y_n: B D C A B B

Y

LCS: Optimal Substructure

- **Case3: $x_m \neq y_n$ and Z = LCS(X,Y)**
  - If $x_m \neq y_n$ and $z_k \neq y_n$, then $Z = LCS(X, Y_{n-1})$
    - Proof is similar to the previous case

if $z_k \neq y_n \rightarrow Z = cs(X, Y_{n-1})$

Z is a longest CS of (X, Y_{n-1}) if not we replace it with the longest and got better answer for $LCS(X, Y)$

X_m: A B C B D A D (X)

Y_n: B D C A B B (Y_{n-1})

LCS: Optimal Substructure

- **Optimal substructure**
  - An optimal solution to a problem (instance) contains optimal solutions to subproblems.
- If $z = LCS(x, y)$, then any prefix of z is an LCS of a prefix of x and a prefix of y
- then there are three cases
  - If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$
  - If $x_m \neq y_n$, then
    - $z_k \neq x_m$ and $Z_k = LCS(X_{m-1}, Y_n)$
    - $z_k \neq y_n$ and $Z_k = LCS(X, Y_{n-1})$

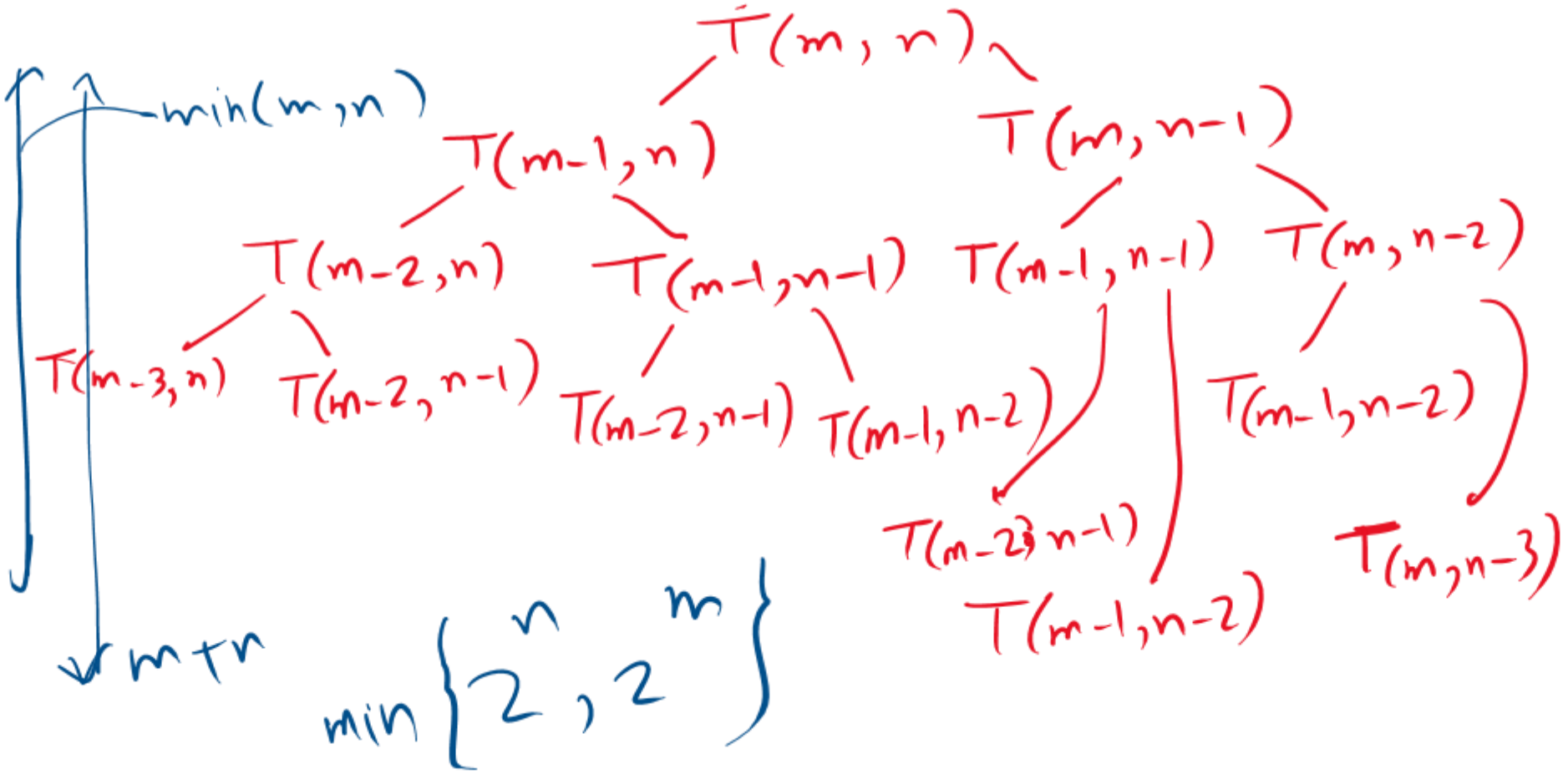
LCS: Recursive Implementation

```
def LCS(X, Y, i, j)
    if i==0 or j==0
        return 0
    else if x[i] = y[j]
        return LCS(X, Y, i-1, j-1) + 1
    else
        return max{LCS(x, y, i-1, j), LCS(x, y,i,j-1)}
```

Worse case: $x[i] \neq y[j]$, in which case the algorithm evaluates two subproblems, each with only one parameter decremented.

$$T(m,n) = T(m-1,n) + T(m,n-1) + O(1)$$

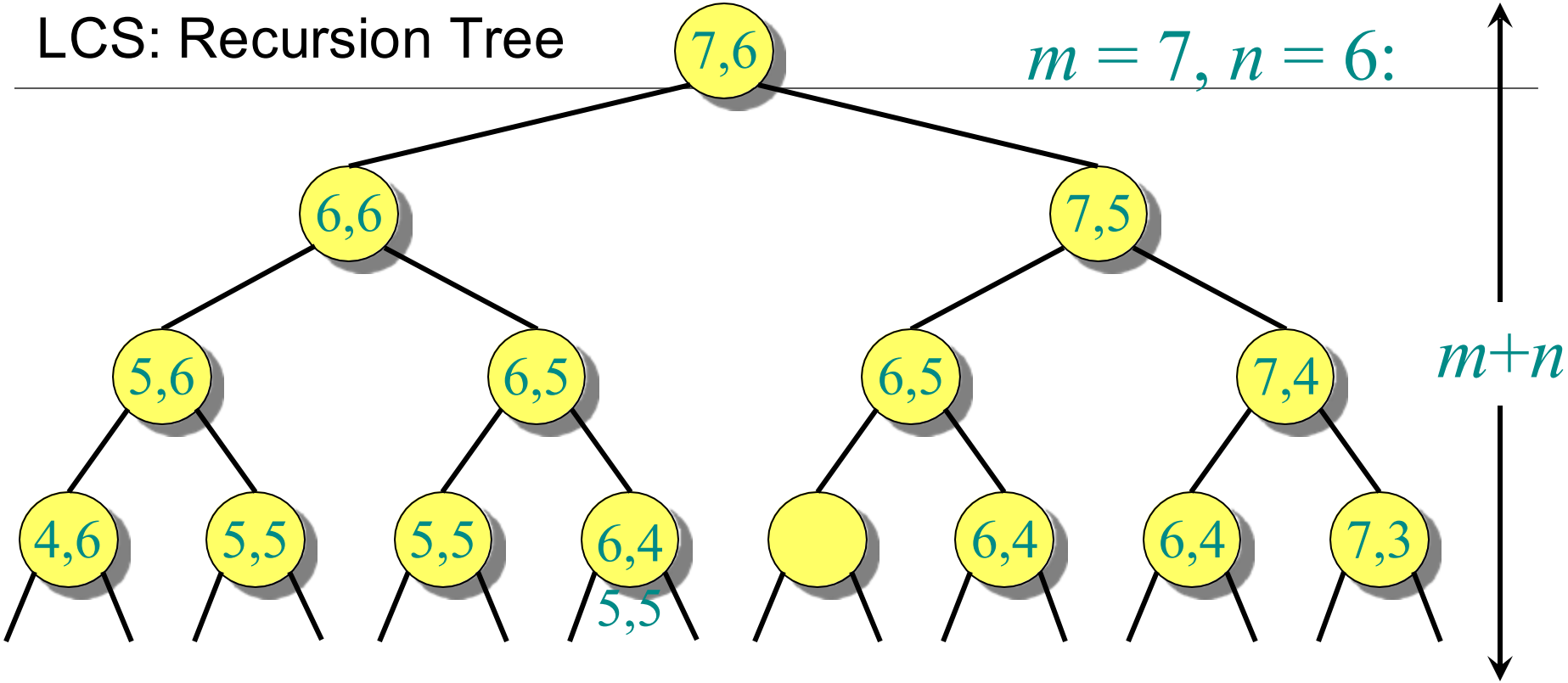
Recursive Implementation: Runtime Analysis



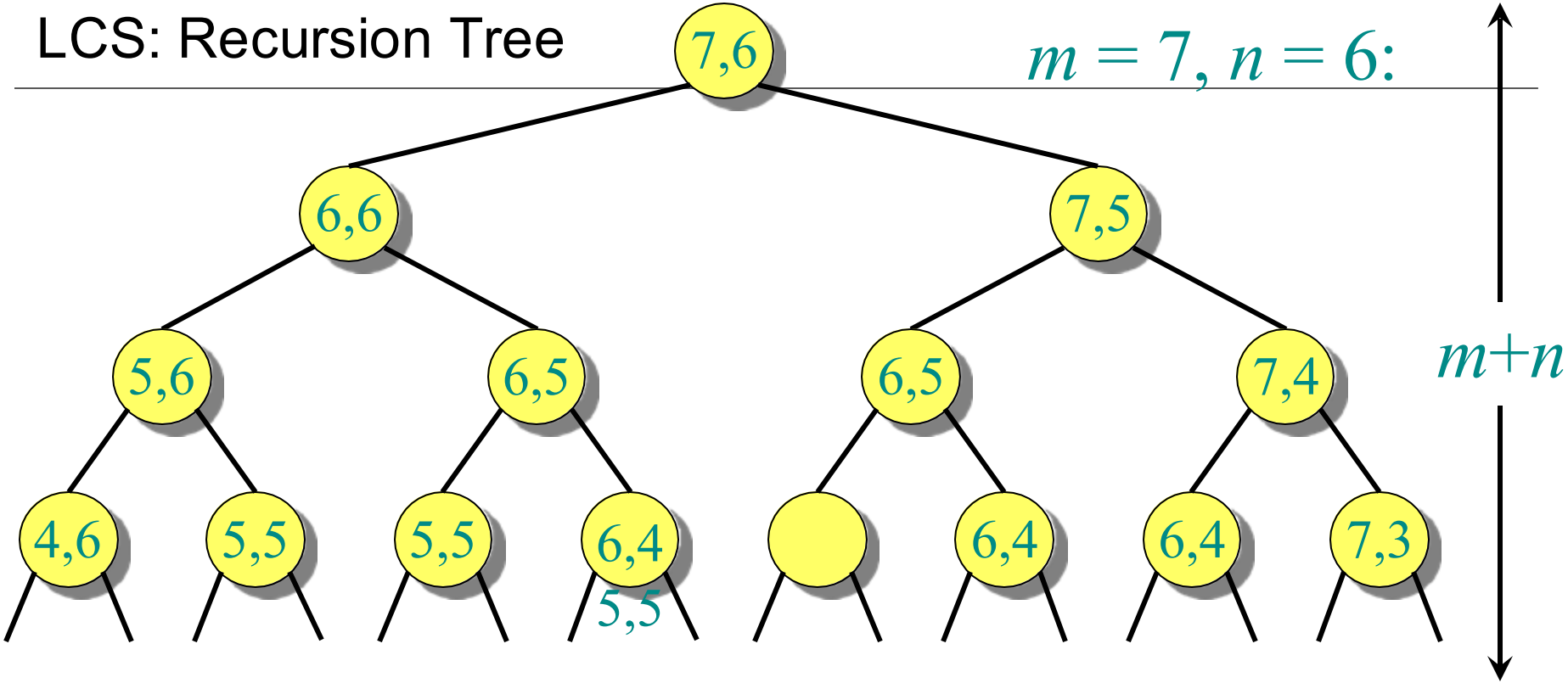
LCS: Overlapping subproblems

- LCS has overlapping subproblem property
  - recursive solution contains a “small” number of distinct subproblems repeated many times.
- The number of distinct LCS subproblems for two strings of lengths *m* and *n* is only *mn*

LCS: Recursion Tree

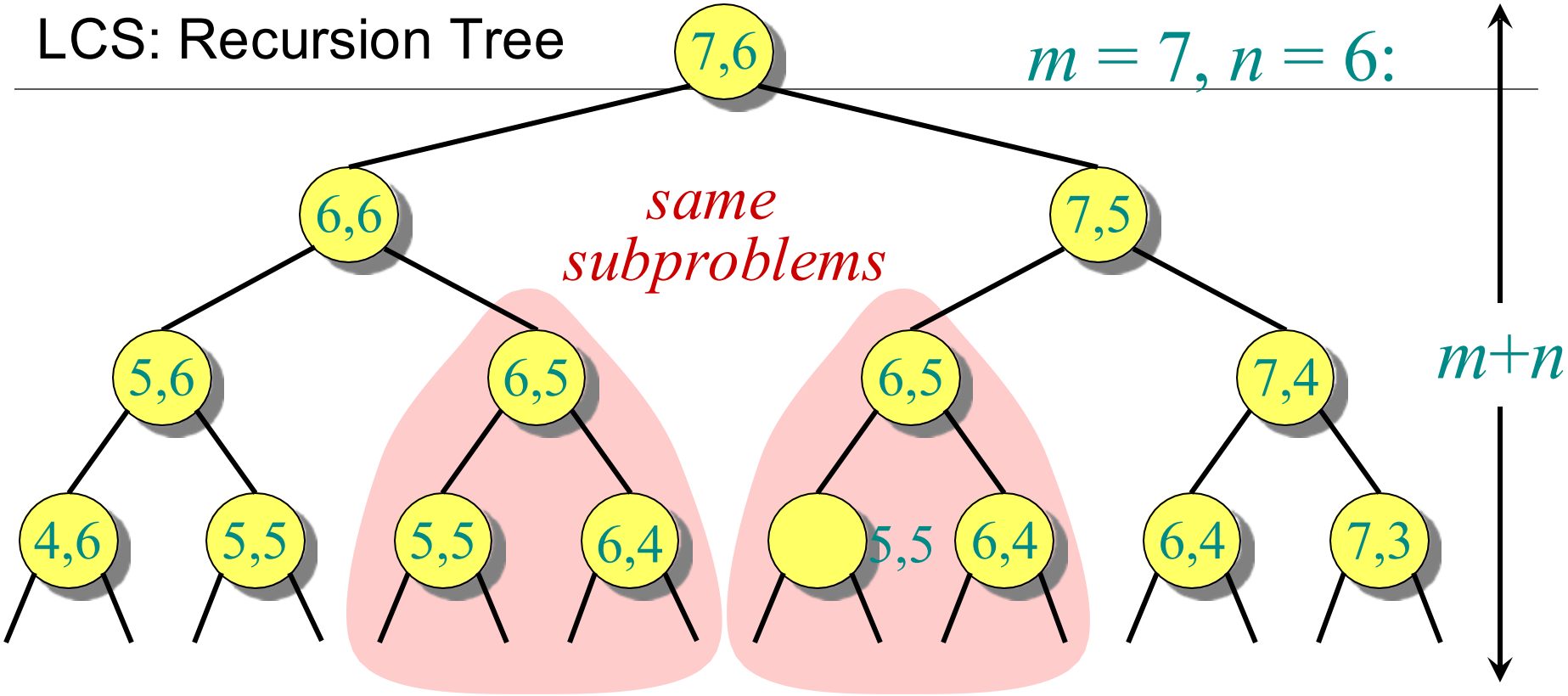


LCS: Recursion Tree



Height = $m + n$ ⇒ work potentially exponential.' but we're solving subproblems already solved!

LCS: Recursion Tree



Height = $m + n$ ⇒ work potentially exponential.' but we're solving subproblems already solved!

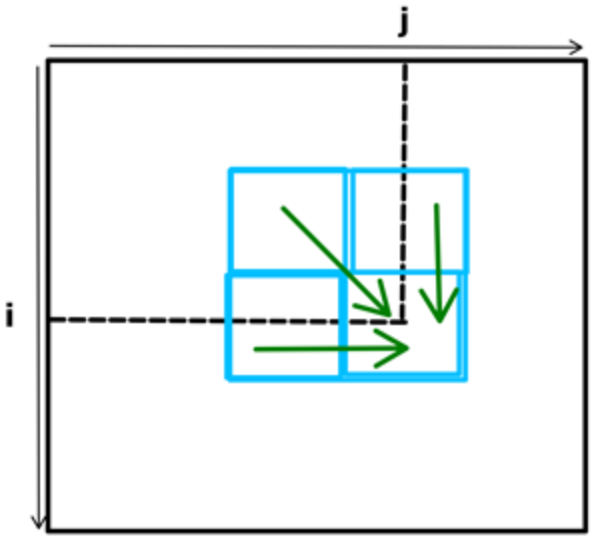
LCS: Bottom-up Solution

$$C[i,j] = \begin{cases} C[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max\{C[i-1, j], C[i, j-1]\} & \text{if } x_i \neq y_j \\ 0 & \text{if } i=0 \text{ or } j=0 \end{cases}$$

$C[i,j]$ = length of the LCS of $X_i = Y_j$

$C[i, 0] = C[0, j]$ for all i,j

LCS: Bottom-up solution

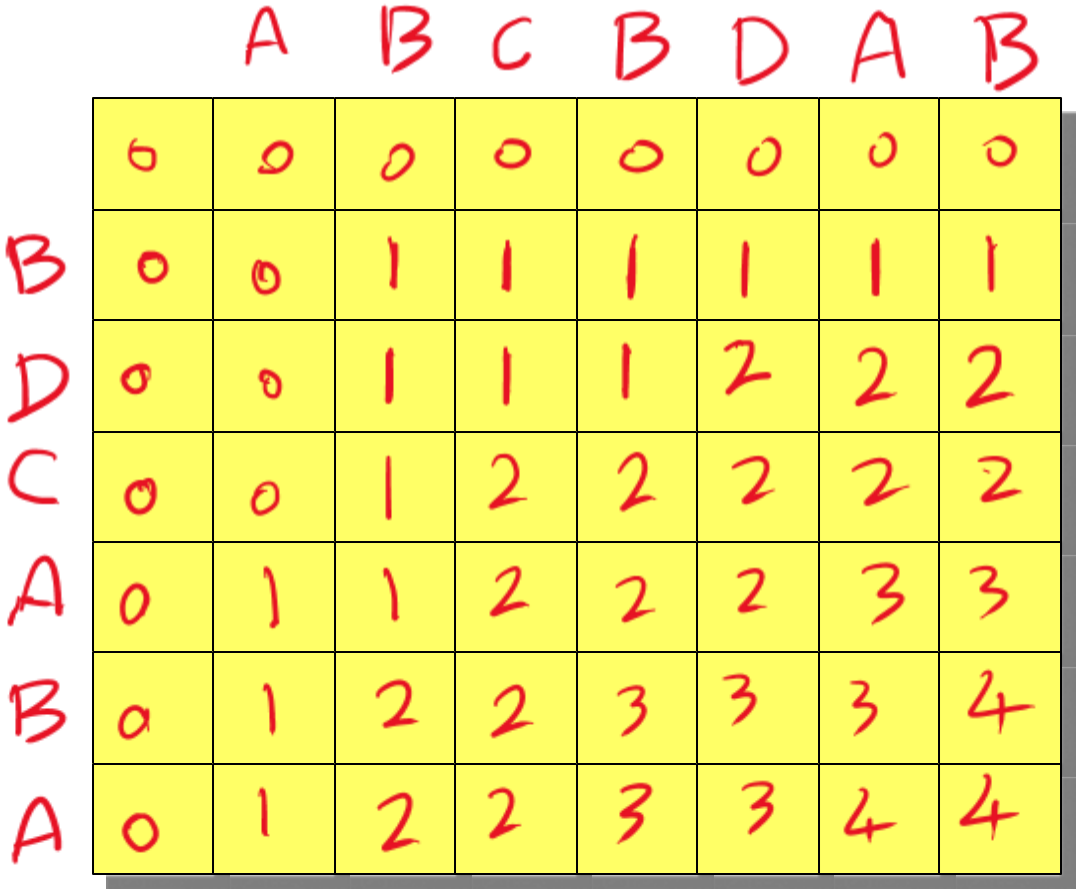



```
def LCS-bottom-up(X,Y,m,n)
    C[0,0]=0
    for i=1 to m
        C[i,0]=0
    for j=1 to n
        C[0,j]=0
    for i=1 to m
        for j=1 to n
            if X[i] = Y[j]
                C[i,j] = C[i-1, j-1] + 1
              else
                C[i,j] = max{ C[i-1, j], C[i, j-1]
      return C[m, n]
```

- **Evaluation order:**
  - Filling in the cells from the top-most corner by rows will ensure that these values will be ready before computing C [i, j].
  - Our answer for the length of an LCS will be found in C [n, m]
- **Runtime:** Θ(nm) time
  - nm subproblems, constant time each
- **Space Complexity:** Θ(n.m)

LCS: Example

How to find the longest common sequence using this table?



		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

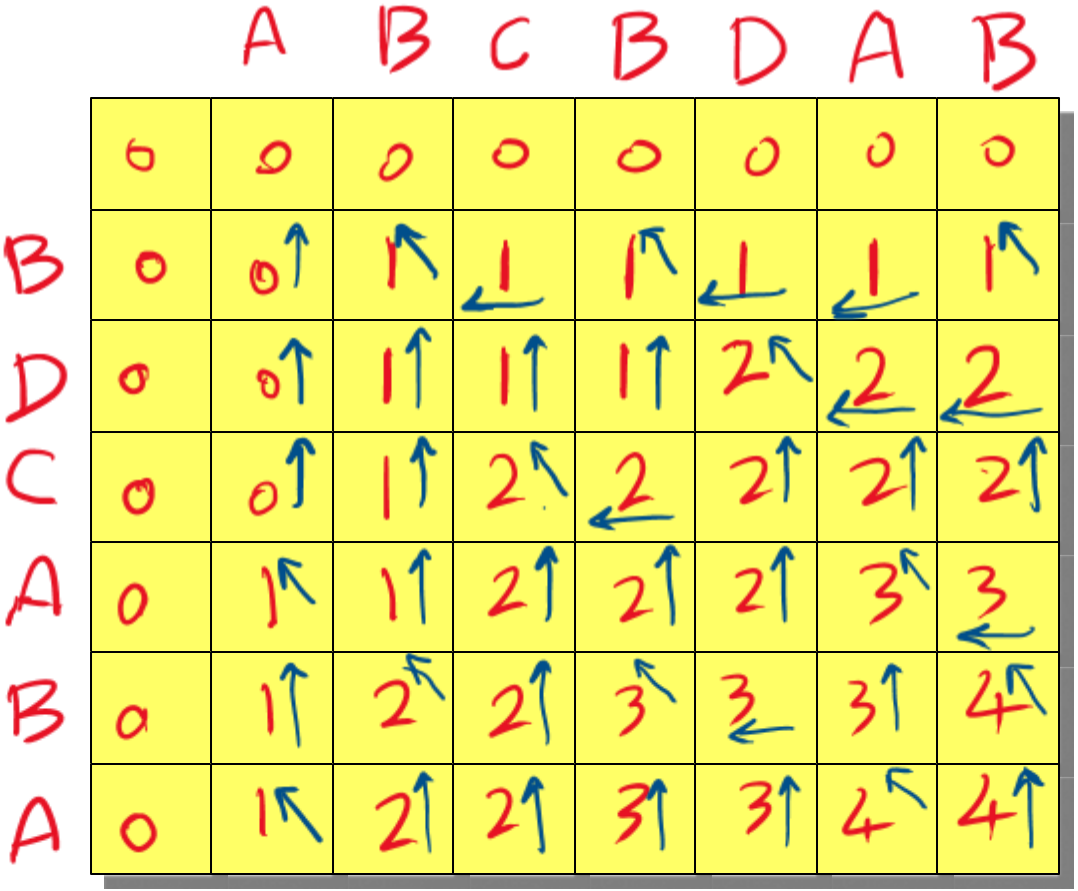
LCS: Bottom-up solution: Actual LCS

- Note that the previous pseudo-code only computed the length of the LCS.
- To get the actual LCS:
- For each entry, keep a backpointer S[i, j] to one of (i − 1, j − 1), (i − 1, j), or (i, j − 1) (depending on which of the three choices determined C[i, j]).
- S[i, j] = "↖" if C[i, j] = 1 + C[i − 1, j − 1]
- S[i, j] = "↑" if C[i, j] = C[i − 1, j]
- S[i, j] = "←" left if C[i, j] = C[i, j − 1]

```
def LCS-bottom-up(X,Y,m,n)
            C[0,0]=0    S[0,0]=0
            for i=1 to m
                C[i,0]=0
            for j=1 to n
                C[0,j]=0
            for i=1 to m
              for j=1 to n
                      if x[i] = y[j]
                         C[i,j] = C[i-1, j-1] + 1
                         S[i,j] = "↖"
                      elif C(i-1, j) ≥ C(i,j-1)
                         C[i,j] = C[i-1, j]
                         S[i,j] = "↑"
                      else:
                         C[i,j] = C[i, j-1]
                         S[i,j] = "←"
      return C[m, n]
```

LCS: Example

How to find the longest common sequence using this table?

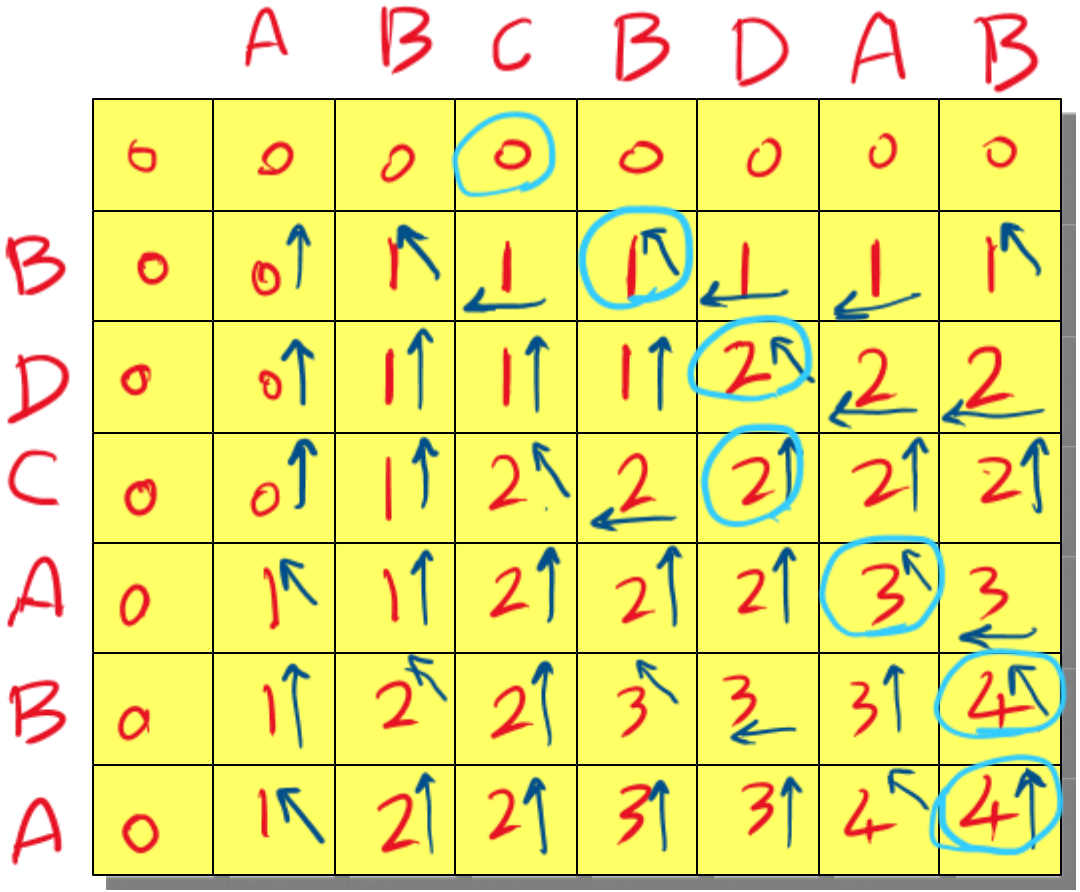


LCS: Example

```
def print-LCS(S, X, i, j)
    if i=0 or j ==0
            return
     if S[i,j] == "↖"
         print-LCS(S, X, i-1, j-1)
         print xi

      elseif S[i,j] == "↑"
            print-LCS(S,X,i-1,j)
      else
            print-LCS(S,X,i,j-1)
```

Runtime: O(m+n) since it decrements at least one of i and j in each recursive call

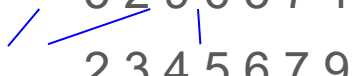


LCS and Longest Increasing Subsequence

- Longest common subsequence solves the problem of Longest increasing subsequence

L: 5 2 9 6 3 7 4

S: 2 3 4 5 6 7 9



- increasing subsequence of length 3
- Note: S = sorted L
- **Claim**: Longest increasing subsequence of L = longest common subsequence of **L** and **S**